

Module 1

Introductory concepts of OOP:

We define “Object Oriented Programming as an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand.”

Programming paradigms

Paradigm can also be termed as method to solve some problem or do some task. Programming paradigm is an approach to solve problem using some programming language or also we can say it is a method to solve a problem using tools and techniques that are available to us following some approach. There are lots for programming language that are known but all of them need to follow some strategy when they are implemented and this methodology/strategy is paradigms.

Imperative programming paradigm: It is one of the oldest programming paradigm. It features close relation to machine architecture. It works by changing the program state through assignment statements. It performs step by step task by changing state. The main focus is on how to achieve the goal. The paradigm consist of several statements and after execution of all the result is stored. Examples of Imperative programming paradigm are C ,Fortran, Basic
Imperative programming is divided into three broad categories: Procedural, OOP and parallel processing. These paradigms are as follows:

- Procedural programming paradigm –
This paradigm emphasizes on procedure in terms of under lying machine model. There is no difference in between procedural and imperative approach. It has the ability to reuse the code and it was boon at that time when it was in use because of its reusability.
Examples of Procedural programming paradigm: C ,C++,Java ,ColdFusion,Pascal
- Object oriented programming –
The program is written as a collection of classes and object which are meant for communication. The smallest and basic entity is object and all kind of computation is performed on the objects only. More emphasis is on data rather procedure. It can handle almost all kind of real life problems which are today in scenario.
 - Examples of Object Oriented programming paradigm are Simula : first OOP language, Java ,C++ (developed by Bjarne Stroustrup),Objective-C ,Visual Basic .NET ,Python ,Ruby ,Smalltalk
- Parallel processing approach –
Parallel processing is the processing of program instructions by dividing them among multiple processors. A parallel processing system posses many numbers of processor with the objective of running a program in less time by dividing them. This approach seems to be like divide and conquer. Examples are NESL (one of the oldest one) and C/C++ also supports because of some library function.

2. Declarative programming paradigm:

It is divided as Logic, Functional, Database. In computer science the *declarative programming* is a style of building programs that expresses logic of computation without talking about its control flow. It often considers programs as theories of some logic.It may simplify writing parallel programs. The focus is on what needs to be done rather how it should be done basically emphasize on what code is actually doing. It just declares the result we want rather how it has be produced. This is the only difference between imperative (how to do) and declarative (what to do) programming paradigms.

- **Logic programming paradigms –**

It can be termed as abstract model of computation. It would solve logical problems like puzzles, series etc. In logic programming we have a knowledge base which we know before and along with the question and knowledge base which is given to machine, it produces result. In normal programming languages, such concept of knowledge base is not available but while using the concept of artificial intelligence, machine learning we have some models

In logical programming the main emphasize is on knowledge base and the problem. The execution of the program is very much like proof of mathematical statement, e.g., Prolog

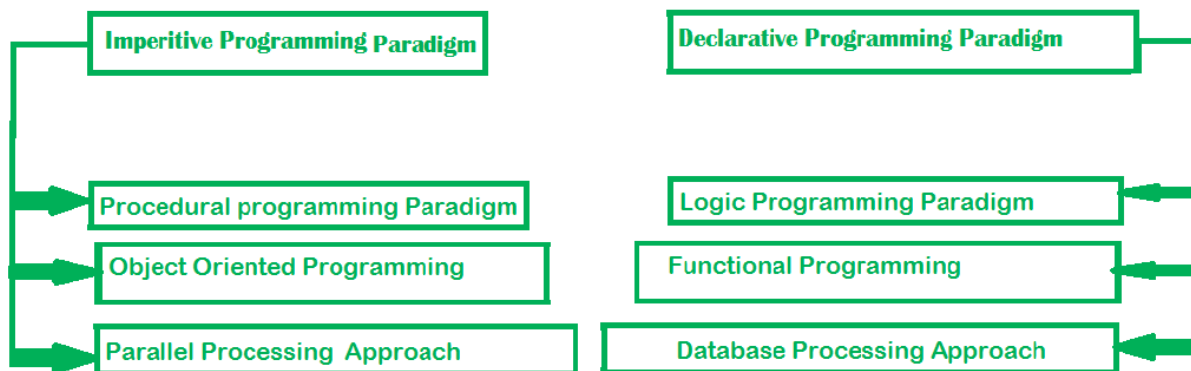
- **Functional programming paradigms –**

The functional programming paradigms has its roots in mathematics and it is language independent. The key principle of this paradigms is the execution of series of mathematical functions. Some of the languages like perl, javascript mostly uses this paradigm.

- **Database/Data driven programming approach –**

This programming methodology is based on data and its movement. Program statements are defined by data rather than hard-coding a series of steps. A database program is the heart of a business information system and provides file creation, data entry, update, query and reporting functions. There are several programming languages that are developed mostly for database application. For example SQL.

Programming Paradigms



Preface to Object Oriented Programming

Key Concepts of OOP: Objects, Classes, Data Abstraction, Encapsulation, Inheritance, Polymorphism, Dynamic Binding, Message Passing (Refer notebook)

Advantages of OOP

(Refer notebook)

C++ Basics:

Parts of C++ Programs

Documentation
Link Section
Definition Section
Global Declaration Section
Function definition Section
Main Function

Skeleton of C Program

➤ **Documentation Section:**

- This section comes first and is used to document the logic of the program that the programmer going to code.
- It can be also used to write for purpose of the program.
- Whatever written in the documentation section is the comment and is not compiled by the compiler.
- Documentation Section is optional since the program can execute without them.

➤ **Linking Section:**

The linking section contains two parts:

a. **Header Files:**

- Generally, a program includes various programming elements like built-in functions, classes, keywords, constants, operators, etc. that are already defined in the standard C++ library.
- In order to use such pre-defined elements in a program, an appropriate header must be included in the program.
- Standard headers are specified in a program through the preprocessor directive #include. In Figure, the iostream header is used. When the compiler processes the instruction #include<iostream>, it includes the contents of the stream in the program. This enables the programmer to use standard input, output, and error facilities that are provided only through the standard streams defined in <iostream>. These standard streams process data as a stream of characters, that is, data is read and displayed in a continuous flow. The standard streams defined in <iostream> are listed here.

#include<iostream>

b. **Namespaces:**

- A namespace permits grouping of various entities like classes, objects, functions, and various C++ tokens, etc. under a single name.
- Any user can create separate namespaces of its own and can use them in any other program.
- In the below snippets, **namespace std** contains declarations for cout, cin, endl, etc. statements. using namespace std;
- Namespaces can be accessed in multiple ways:
 - using namespace std;
 - using std :: cout;

➤ **Definition Section:**

- It is used to declare some constants and assign them some value.
- In this section, anyone can define your own datatype using primitive data types.

➤ **Global Declaration Section:**

- Here, the variables and the class definitions which are going to be used in the program are declared to make them global.
- The scope of the variable declared in this section lasts until the entire program terminates.
- These variables are accessible within the user-defined functions also.

➤ **Function Declaration Section:**

- It contains all the functions which our main functions need.
- Usually, this section contains the User-defined functions.
- This part of the program can be written after the main function but for this, write the function prototype in this section for the function which for you are going to write code after the main function.

➤ **Main Function:**

- The main function tells the compiler where to start the execution of the program. The execution of the program starts with the main function.
- All the statements that are to be executed are written in the main function.
- The compiler executes all the instructions which are written in the curly braces {} which encloses the body of the main function.
- Once all instructions from the main function are executed, control comes out of the main function and the program terminates and no further execution occur.

Classes in C++

Class is a collection of data members and member functions.

Syntax:

Class classname

Private:

Variable declarations;

Function declarations;

Public:

Variable declarations;

Function declarations;

Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, and mileage are their properties.

A Class is a user-defined data type that has data members and member functions.

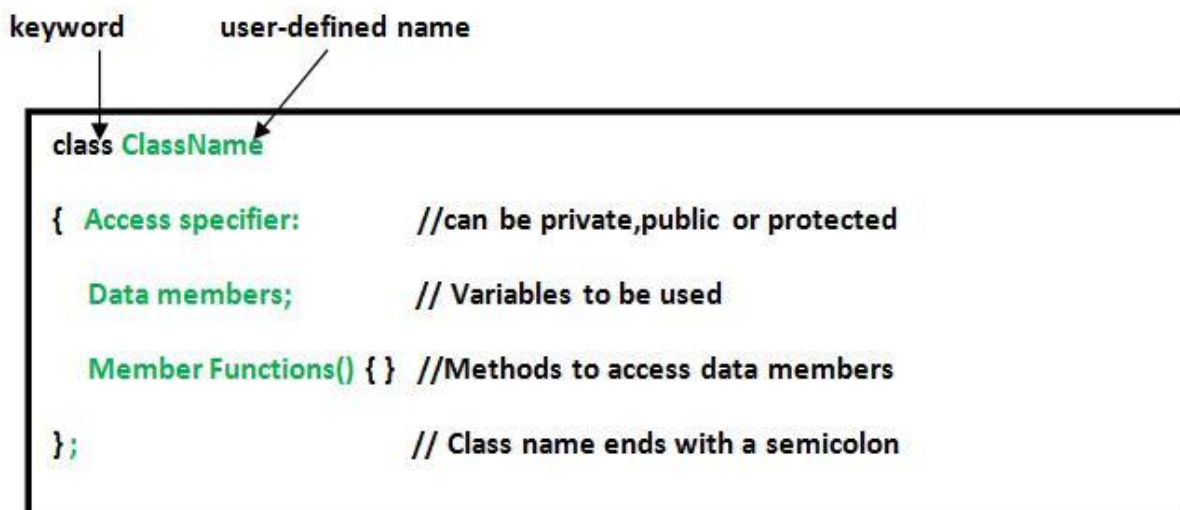
Data members are the data variables and member functions are the functions used to manipulate these variables together, these data members and member functions define the properties and behavior of the objects in a Class.

In the above example of class Car, the data member will be speed limit, mileage, etc, and member functions can be applying brakes, increasing speed, etc.

An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using the keyword class followed by the name of the class. The body of the class is defined inside the curly brackets and terminated by a semicolon at the end.



Declaring Objects

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax

ClassName ObjectName;

Accessing data members and member functions: The data members and member functions of the class can be accessed using the dot('.') operator with the object. For example, if the name of the object is obj and you want to access the member function with the name printName() then you will have to write obj.printName().

```
class item
{
    int number;           // variables declaration
    float cost;           // private by default
public:
    void getdata(int a, float b); // functions declaration
    void putdata(void);           // using prototype
}; // ends with semicolon
```

public, private and protected keywords

Public: All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

Private: The class members declared as private can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class.

Protected: The protected access modifier is similar to the private access modifier in the sense that it can't be accessed outside of its class unless with the help of a friend class. The difference is that the class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

(refer example in notebook)

Defining Member Functions

There are 2 ways to define a member function:

Inside class definition

Outside class definition

To define a member function outside the class definition we have to use the scope resolution:: operator along with the class name and function name.

Refer notebook for examples

Characteristics of Member Functions

- Several different classes can use the same function name. The 'membership label' will resolve their scope.
- Member functions can access the private data of the class. A non-member function cannot do so. (However, an exception to this rule is a *friend* function discussed later.)
- A member function can call another member function directly, without using the dot operator.

Array of Objects

An array of objects is a collection of objects of the same class type that are stored in contiguous memory locations. Since each item in the array is an instance of the class, each one's member variables can have a unique value. This makes it possible to manage and handle numerous objects by storing them in a single data structure and giving them similar properties and behaviours.

No memory or storage is allocated when a class is defined; only the object's specification is defined. We must create objects in order to use the data and access the class's defined functions.

We can think of array of objects as a single variable that can hold multiple values. Each value is stored in a separate element of the array, and each element can be accessed by its index

Syntax:

```
ClassName ObjectName[number of objects];
```

Refer example in notebook

Objects as Function Argument

Like any other data type, an object may be used as a function argument. This can be done in two ways:

- A copy of the entire object is passed to the function.
- Only the address of the object is transferred to the function.

The first method is called *pass-by-value*. Since a copy of the object is passed to the function, any changes made to the object inside the function do not affect the object used to call the function. The second method is called *pass-by-reference*. When an address of the object is passed, the called function works directly on the actual object used in the call. This means that any changes made to the object inside the function will reflect in the actual object. The pass-by reference method is more efficient since it requires to pass only the address of the object and not the entire object.

Refer example in notebook